

Department Of Computer Studies

B. Sc. (CA & IT) Honours- III

U13A2OOP- OBJECT ORIENTED CONCEPTS AND PROGRAMMING

Unit3: - Inheritance and Interface

Inheritance

Inheritance is the process by which one class acquires the properties (data members) and behaviors (methods) of another class. **Inheritance** is the process by which one class acquires the properties (data members) and behaviors (methods) of another class.

Inheritance is a mechanism that allows one class to reuse the fields and methods of another class.

Syntax

```
class ParentClass
{
    // Data Members

    // Methods
}
class ChildClass extends ParentClass
{
    // Additional Members
}
```

Advantages

1. Code Reusability

Same code can be reused.

2. Less Coding

No need to rewrite methods.

3. Easy Maintenance

Changes in parent class automatically affect child class.

4. Faster Development

Less time required.

5. Supports Runtime Polymorphism

Method overriding is possible.

Disadvantages

- Tight coupling between classes

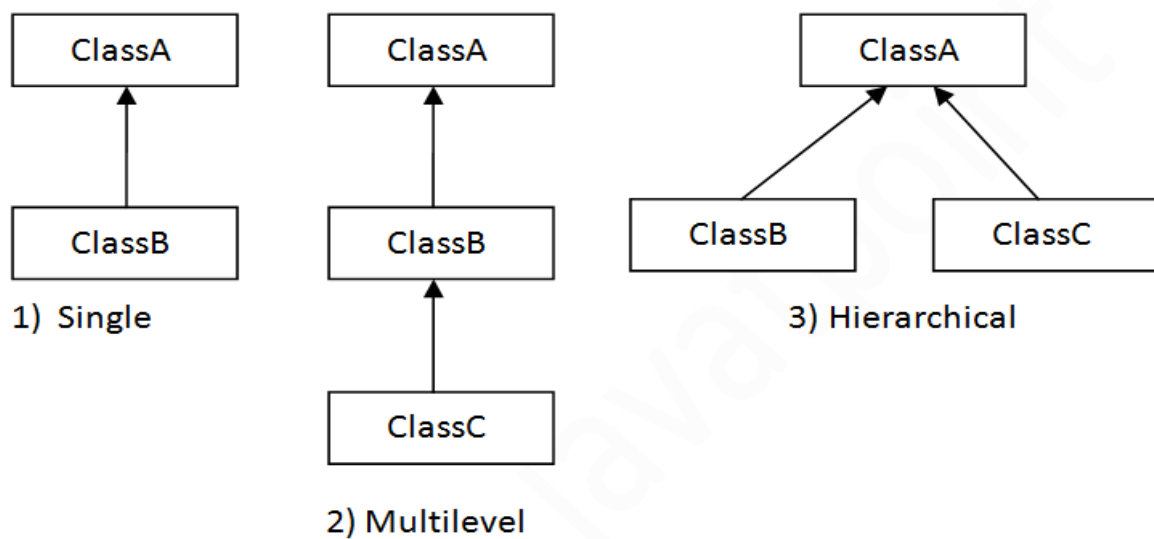
- Complex hierarchy may be difficult to understand
- Incorrect inheritance can reduce flexibility

Types of Inheritance in Java

Java supports the following types:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance

Java **does not support Multiple Inheritance with classes**, but it supports it using **interfaces**.



When a class inherits another class, it is known as a single inheritance.

Example 1 – Single Inheritance

```
class Animal
{
    void eat()
    {
        System.out.println("Animal is Eating");
    }
}

class Dog extends Animal
{
    void bark()
    {
```

```

        System.out.println("Dog is Barking");
    }
}
public class InheritanceDemo
{
    public static void main(String args[])
    {
        Dog d = new Dog();

        d.eat();
        d.bark();
    }
}

```

Multilevel Inheritance Example

Multilevel Inheritance is a type of inheritance in which a class inherits another class, and then a third class inherits the second class.

Multilevel inheritance is the process of deriving a class from another derived class.

Syntax

```

class A
{
}

class B extends A
{
}

class C extends B
{
}

class Person
{
    void displayName()
    {

```

```

        System.out.println("Name : Raj");
    }
}
class Student extends Person
{
    void displayCourse()
    {
        System.out.println("Course : B.Sc. AI & ML");
    }
}
class Result extends Student
{
    void displayMarks()
    {
        System.out.println("Marks : 450");
    }
}
public class StudentDemo
{
    public static void main(String args[])
    {
        Result r = new Result();
        r.displayName();
        r.displayCourse();
        r.displayMarks();
    }
}

```

Hierarchical Inheritance

Hierarchical Inheritance is a type of inheritance in which **two or more child classes inherit from a single parent class**.

Hierarchical inheritance is the process where one parent class is inherited by multiple child classes.

Syntax

```
class Parent
{
    // Data Members
    // Methods
}
class Child1 extends Parent
{
}
class Child2 extends Parent
{
}
class Child3 extends Parent
{
}
```

Example

```
class Animal
{
    void eat()
    {
        System.out.println("Animal is Eating");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog is Barking");
    }
}
```

```

class Cat extends Animal
{
    void meow()
    {
        System.out.println("Cat is Meowing");
    }
}

public class HierarchicalDemo
{
    public static void main(String args[])
    {
        Dog d = new Dog();
        d.eat();
        d.bark();
        System.out.println();
        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}

```

Use of super Keyword

- The super keyword is one of the most important keywords in Java. It is used in inheritance to refer to the immediate parent (super) class.
- When a child class inherits a parent class, both classes may have constructors, variables, or methods with the same name. The super keyword helps the child class access the parent class members.
- super refers to the immediate parent class object.
- The super keyword is a reference variable used to refer to the immediate parent class object.

It is mainly used to:

1. Access parent class variables.

2. Call parent class methods.
3. Invoke the parent class constructor.

Syntax

Access Parent Variable

```
super.variableName;
```

Call Parent Method

```
super.methodName();
```

Call Parent Constructor

```
super();
```

Use 1: Access Parent Class Variable

If the parent and child classes have variables with the same name, super accesses the parent class variable.

```
class Animal
{
    String name = "Animal";
}

class Dog extends Animal
{
    String name = "Dog";
    void display()
    {
        System.out.println("Child Name : " + name);
        System.out.println("Parent Name : " + super.name);
    }
}

public class SuperVariableDemo
{
    public static void main(String args[])
    {
```

```
Dog d = new Dog();  
d.display();  
}  
}
```

Use 2: Call Parent Class Method

The super keyword can call the parent class method when it is overridden in the child class.

```
class Animal  
{  
    void sound()  
    {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal  
{  
    void sound()  
    {  
        System.out.println("Dog Barks");  
    }  
    void display()  
    {  
        super.sound();  
        sound();  
    }  
}  
  
public class SuperMethodDemo  
{  
    public static void main(String args[])  
    {
```



```
        Dog d = new Dog();  
        d.display();  
    }  
}
```

Use 3: Call Parent Class Constructor

The `super()` keyword is used to invoke the constructor of the parent class.

```
class Animal  
{  
    Animal()  
    {  
        System.out.println("Animal Constructor");  
    }  
}  
  
class Dog extends Animal  
{  
    Dog()  
    {  
        super();  
        System.out.println("Dog Constructor");  
    }  
}  
  
public class SuperConstructorDemo  
{  
    public static void main(String args[])  
    {  
        Dog d = new Dog();  
    }  
}
```

Write a Java program to demonstrate the use of the super keyword.

```
class Vehicle
{
    String brand = "Toyota";

    void start()
    {
        System.out.println("Vehicle Started");
    }
}

class Car extends Vehicle
{
    String brand = "BMW";

    void show()
    {
        System.out.println("Child Brand : " + brand);
        System.out.println("Parent Brand : " + super.brand);
        super.start();
    }
}

public class PracticalSuper
{
    public static void main(String args[])
    {
        Car c = new Car();
        c.show();
    }
}
```

method overriding

Method overriding occurs when a child class provides its own implementation of a method that is already defined in its parent class.

Method Overriding is a feature in Java where a subclass defines a method with the same name, same parameters, and same return type as a method in its superclass.

Syntax

```
class Parent
{
    void display()
    {
        // Parent implementation
    }
}

class Child extends Parent
{
    @Override
    void display()
    {
        // Child implementation
    }
}
```

Example 1 – Basic Method Overriding

```
class Animal
{
    void sound()
    {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal
{
}
```

```

@Override
void sound()
{
    System.out.println("Dog Barks");
}
}

public class MethodOverridingDemo
{
    public static void main(String args[])
    {
        Dog d = new Dog();
        d.sound();
    }
}

```

Abstract Class

An **abstract class** is a class that is declared using the **abstract** keyword. It **cannot be instantiated** (you cannot create its object).

Abstract Method

An **abstract method** is a method declared using the **abstract** keyword **without a method body**. It must be implemented by the subclass.

Syntax

```

abstract class ClassName
{
    // Variables
    // Constructors
    // Normal methods
    // Abstract methods
}

```

Features of Abstract Class

- Declared using the abstract keyword.

- Cannot create an object directly.
- May contain abstract and normal methods.
- May contain constructors and variables.
- Used as a base class.
- Supports abstraction.

Features of Abstract Method

- Declared using the abstract keyword.
- Does not have a method body.
- Ends with a semicolon (;).
- Must be implemented by the child class.

Example 1 – Abstract Class

```
abstract class Vehicle
```

```
{
    abstract void start();
    void stop()
    {
        System.out.println("Vehicle Stopped");
    }
}
```

```
class Car extends Vehicle
```

```
{
    void start()
    {
        System.out.println("Car Started");
    }
}
```

```
public class AbstractDemo
```

```
{
    public static void main(String args[])
```

```
{  
    Car c = new Car();  
    c.start();  
    c.stop();  
}  
}
```

Another example

abstract class Person

```
{  
    abstract void work();  
    void display()  
    {  
        System.out.println("Person Details");  
    }  
}
```

class Student extends Person

```
{  
    void work()  
    {  
        System.out.println("Student is Studying");  
    }  
}
```

public class PracticalAbstract

```
{  
    public static void main(String args[])  
    {  
        Student s = new Student();  
        s.display();  
        s.work();  
    }  
}
```

```
}  
}
```

Use of final modifier

The **final** keyword is one of the most important keywords in Java. It is used to **restrict modifications** to variables, methods, and classes.

The final keyword makes the code **secure, reliable**, and **prevents accidental changes**.

It can be applied to:

- Variables
- Methods
- Classes

Once a variable, method, or class is declared as final, it cannot be changed according to its type.

Uses of final Keyword

The final keyword can be used with:

1. Final Variable

A **final variable** can be assigned a value **only once**. After initialization, its value cannot be changed.

```
public class FinalVariableDemo  
{  
    public static void main(String args[])  
    {  
        final int AGE = 20;  
        System.out.println("Age : " + AGE);  
        // AGE = 25; // Error  
    }  
}
```

2. Final Method

A **final method** cannot be **overridden** by a subclass.

```
class Animal  
{
```

```

    final void eat()
    {
        System.out.println("Animal is Eating");
    }
}

class Dog extends Animal
{
    /*
    void eat()
    {
        System.out.println("Dog is Eating");
    }
    */
    void bark()
    {
        System.out.println("Dog is Barking");
    }
}

public class FinalMethodDemo
{
    public static void main(String args[])
    {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}

```

3. Final Class

A **final class** cannot be inherited.


```

final class Animal
{
    void display()
    {
        System.out.println("Animal Class");
    }
}

/*
class Dog extends Animal
{
}
*/

public class FinalClassDemo
{
    public static void main(String args[])
    {
        Animal a = new Animal();
        a.display();
    }
}

```

Defining and implementing Interface

- An **Interface** in Java is a blueprint or contract that defines a set of methods that a class must implement.
- An interface tells a class **what to do**, but traditionally does not specify **how to do it**.
- An **interface** is a reference type in Java that contains a contract of methods and constants that implementing classes must follow.
- An interface is declared using the **interface** keyword.

Syntax

```
interface InterfaceName
```

```
{  
    // methods  
    // constants  
}
```

A class implements an interface using the implements keyword.

```
class ClassName implements InterfaceName
```

```
{  
    // implementation of interface methods  
}
```

Example:-

```
interface Animal
```

```
{  
    void sound();  
    void eat();  
}
```

```
class Dog implements Animal
```

```
{  
    public void sound()  
    {  
        System.out.println("Dog Barks");  
    }  
    public void eat()  
    {  
        System.out.println("Dog Eats");  
    }  
}
```

```
public class InterfaceDemo
```

```
{  
    public static void main(String args[])
```

```
{  
    Dog d = new Dog();  
    d.sound();  
    d.eat();  
}  
}
```

Multiple Inheritance Using Interfaces

Java does **not** support multiple inheritance using classes.

For example, this is not allowed:

class C extends A, B

```
{  
}
```

Example:-

interface Printable

```
{  
    void print();  
}
```

interface Showable

```
{  
    void show();  
}
```

class Demo implements Printable, Showable

```
{  
    public void print()  
    {  
        System.out.println("Printing");  
    }  
    public void show()  
    {  
        System.out.println("Showing");  
    }  
}
```

```
}  
  
public class MultipleInterfaceDemo  
{  
    public static void main(String args[])  
    {  
        Demo d = new Demo();  
        d.print();  
        d.show();  
    }  
}
```

Interface Extending Another Interface

An interface can extend another interface using the **extends** keyword.

```
interface Animal  
{  
    void eat();  
}  
  
interface Dog extends Animal  
{  
    void bark();  
}  
  
class Puppy implements Dog  
{  
    public void eat()  
    {  
        System.out.println("Puppy Eats");  
    }  
    public void bark()  
    {  
        System.out.println("Puppy Barks");  
    }  
}
```

```
public class InterfaceExtendDemo
{
    public static void main(String args[])
    {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
    }
}
```

Default Interface Methods

From **Java 8**, an interface can contain a default method with a body.

```
interface Vehicle
{
    void start();
    default void stop()
    {
        System.out.println("Vehicle Stopped");
    }
}

class Car implements Vehicle
{
    public void start()
    {
        System.out.println("Car Started");
    }
}

public class DefaultMethodDemo
{
    public static void main(String args[])
    {
```

```
Car c = new Car();  
c.start();  
c.stop();  
}  
}
```

Multithreading

What is a Thread?

A **thread** is a small unit of execution within a program.

A Java program can perform multiple tasks at the same time using multiple threads.

What is Multithreading?

Multithreading is the process of executing multiple threads concurrently within a single program.

Multithreading allows a program to perform multiple tasks at the same time or in overlapping execution.

Creating Threads in Java

There are two basic ways commonly taught:

1. By extending the Thread class
2. By implementing the Runnable interface

Method 1: Extending the Thread Class

Create a class that extends Thread and override its run() method.

Syntax

```
class MyThread extends Thread  
{  
    public void run()  
    {  
        // Thread task  
    }  
}
```

Example – Thread Class

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread is Running");
    }
}

public class ThreadDemo
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Example – Thread with Loop

```
class NumberThread extends Thread
{
    public void run()
    {
        for(int i = 1; i <= 5; i++)
        {
            System.out.println(i);
        }
    }
}

public class NumberDemo
{
    public static void main(String args[])
```

```
{  
    NumberThread t = new NumberThread();  
    t.start();  
}  
}
```

Method 2: Implementing Runnable

Syntax

class MyTask implements Runnable

```
{  
    public void run()  
    {  
        // Thread task  
    }  
}
```

Example – Runnable Interface

class MyTask implements Runnable

```
{  
    public void run()  
    {  
        System.out.println("Runnable Thread is Running");  
    }  
}
```

public class RunnableDemo

```
{  
    public static void main(String args[])  
    {  
        MyTask obj = new MyTask();  
        Thread t = new Thread(obj);  
        t.start();  
    }  
}
```



```
}  
}
```

Thread Methods

Method	Purpose
start()	Starts a new thread
run()	Contains the task performed by the thread
sleep()	Pauses the current thread for a specified time
join()	Waits for another thread to finish
getName()	Gets thread name
setName()	Sets thread name
currentThread()	Returns currently executing thread
isAlive()	Checks whether a thread is still alive

Write a Java program to create two threads using the Thread class.

```
class EvenThread extends Thread
```

```
{  
    public void run()  
    {  
        for(int i = 2; i <= 10; i = i + 2)  
        {  
            System.out.println("Even : " + i);  
        }  
    }  
}
```

```
class OddThread extends Thread
```

```
{  
    public void run()  
    {
```

```

        for(int i = 1; i <= 9; i = i + 2)
        {
            System.out.println("Odd : " + i);
        }
    }
}

public class ThreadPractical
{
    public static void main(String args[])
    {
        EvenThread e = new EvenThread();
        OddThread o = new OddThread();
        e.start();
        o.start();
    }
}

```

What is Exception Handling?

Exception handling in Java is a mechanism used to **handle runtime errors** so that the normal flow of a program can continue instead of terminating unexpectedly.

An **exception** is an unwanted or unexpected event that occurs during program execution.

Types of Errors in Java

Generally, errors encountered during programming can be classified into:

1. Compile-Time Error

These errors occur when the Java compiler finds a problem in the source code.

Example:

```
int a = 10
```

```
System.out.println(a);
```

The semicolon is missing.

The compiler reports an error.

2. Runtime Error

A runtime error occurs while the program is executing.

Example:

```
int a = 10;
```

```
int b = 0;
```

```
System.out.println(a / b);
```

Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero

try and catch

The most basic exception-handling mechanism uses:

```
try
```

```
{
```

```
    // code that may cause exception
```

```
}
```

```
catch(ExceptionType e)
```

```
{
```

```
    // code to handle exception
```

```
}
```

Example

```
public class ExceptionDemo {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            int a = 10;
```

```
            int b = 0;
```

```
            int result = a / b;
```

```
            System.out.println("Result = " + result);
```

```
        }
```

```
        catch (ArithmeticException e) {
```

```
            System.out.println("Cannot divide by zero.");
```

```

    }
    System.out.println("Program continues...");
}
}

```

Output

Cannot divide by zero.

Program continues...

Here, the exception occurs inside the try block.

The catch block catches the `ArithmeticException` and handles it.

finally Block

The finally block contains statements that normally need to execute whether an exception occurs or not.

Syntax:

```

try {
    // risky code
}
catch(Exception e) {
    // exception handling
}
finally {
    // code that normally executes afterward
}

```

Example

```

public class FinallyDemo {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;

            System.out.println(a / b);

```

```

    }
    catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero.");
    }
    finally {
        System.out.println("Finally block executed.");
    }
}
}

```

Output:

Cannot divide by zero.

Finally block executed.

The finally block is commonly used for **cleanup activities**, such as closing resources.

Multiple catch Blocks

A single try block can have multiple catch blocks.

Example:

```

public class MultipleCatchDemo {
    public static void main(String[] args) {
        try {
            int[] numbers = {10, 20, 30};
            System.out.println(numbers[5]);
        }
        catch (ArithmeticException e) {
            System.out.println("Arithmetic error occurred.");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index.");
        }
        catch (Exception e) {

```

```
        System.out.println("Some other exception occurred.");  
    }  
}  
}
```